
Why Do We Need Weight Decay in Modern Deep Learning?

Francesco D’Angelo*, Maksym Andriushchenko*, Aditya Varre, Nicolas Flammarion

Theory of Machine Learning Lab

EPFL, Lausanne, Switzerland

{francesco.dangelo,maksym.andriushchenko,aditya.varre,nicolas.flammarion}@epfl.ch

Abstract

Weight decay is a broadly used technique for training state-of-the-art deep networks from image classification to large language models. Despite its widespread usage and being extensively studied in the classical literature, its role remains poorly understood for deep learning. In this work, we highlight that the role of weight decay in modern deep learning is different from its regularization effect studied in classical learning theory. For deep networks on vision tasks trained with multipass SGD, we show how weight decay modifies the optimization dynamics enhancing the ever-present implicit regularization of SGD via the *loss stabilization mechanism*. In contrast, for large language models trained with nearly one-epoch training, we describe how weight decay balances the *bias-variance tradeoff* in stochastic optimization leading to lower training loss and improved training stability. Overall, we present a unifying perspective from ResNets on vision tasks to LLMs: weight decay is never useful as an explicit regularizer but instead changes the training dynamics in a desirable way. The code is available at <https://github.com/tml-epfl/why-weight-decay>

1 Introduction

The training of modern neural networks broadly falls into two regimes: *over-training*, which involves multiple passes through a dataset and necessitates effective regularization strategies to avoid overfitting; and *under-training*, characterized by fewer passes due to large amounts of training data and computational constraints (Hoffmann et al., 2022). Modern deep learning unequivocally embodies both training regimes: ResNet architectures on computer vision tasks (He et al., 2016) serve as quintessential examples of the over-training regime, while the training of large language models (Brown et al., 2020) stands as a hallmark of the under-training regime. Despite their differences, both regimes extensively adopt weight decay as a regularization technique, though its effectiveness and role remain subjects of ongoing debate. For the first regime, Zhang et al. (2016) showed that even when using weight decay, neural networks can still fully memorize the data, thus questioning its regularization properties. For the second, regularization is inherently unnecessary as the limited number of passes already prevents overfitting. These considerations raise important questions about the necessity and purpose of weight decay, introducing uncertainty about its widespread usage. To illustrate the effect of weight decay in the two regimes,

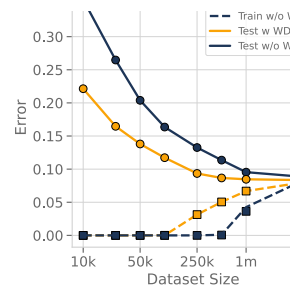


Figure 1: Test error vs. dataset size on CIFAR-10-5m for a fixed number of training iteration. Weight decay is helpful in both: the over-training and the under-training, one-pass regime.

*Equal contribution

we conduct a simple experiment. We train a ResNet18 on subsets of the CIFAR-5m dataset (Nakkiran et al., 2020) with sizes from 10 000 to 5 mln. The computational budget of each training session is fixed to 5 mln iterations, which amounts to a range of passes between 500 and one. In the over-training regime (left in Fig. 1), weight decay does not prevent the models from achieving zero training error, but its presence still improves the test error. Attempting to explain this generalization benefit, recent works (Li & Arora, 2019; Li et al., 2020) bring forth the hypothesis that it is inadequate to think about weight decay as a capacity constraint since it still bears an effect on the training of scale-invariant models. As a result, understanding the effect of weight decay on the optimization dynamics becomes crucial to understanding generalization. Nevertheless, this line of work heavily relies on an *effective learning rate* (ELR) which only emerges as a consequence of scale-invariance and therefore does not apply to general architectures. In the under-training regime (right in Fig. 1), where the generalization gap vanishes, weight decay seem to facilitate faster training for slightly better accuracy. However, a characterization of the mechanisms through which weight decay impacts the training speed in this regime remains underexplored.

Our work delves into the mechanisms underlying the benefits of weight decay by training established machine learning models in both regimes: ResNet on popular vision tasks (over-training) and Transformer on text data (under-training). Towards this goal, we make the following contributions:

- In the over-training regime, we unveil the mechanism by which weight decay effectively reduces the generalization gap. We demonstrate that combining weight decay with large learning rates enables non-vanishing SGD noise, which through its implicit regularization controls the norm of the Jacobian leading to improved performance. Moreover, our investigation offers a thorough explanation for the effectiveness of employing exponential moving average and learning rate decay in combination with weight decay.
- In the under-training regime, particularly for LLMs trained with one-pass Adam, we confirm experimentally that weight decay does not bring any regularization effect and is simply equivalent to a modified ELR. We explain the training curves commonly observed with weight decay: through this ELR, weight decay better modulates the bias-variance trade-off, resulting in lower loss. Additionally, we show that weight decay has another important practical benefit: enabling stable training with the bfloat16 precision.

1.1 Related work

The concept of employing ℓ_2 weight penalty traces back to studies on the stability of solutions for ill-posed problems (Tikhonov, 1943). It has since been extensively explored in statistics (Foster, 1961; Hoerl, 1962; Hoerl & Kennard, 1970). Krogh & Hertz (1991) present one of the earliest systematic studies on weight decay tailored for *neural networks*. Generalization bounds, such as those by Shalev-Shwartz & Ben-David (2014), suggest that weight decay can be *sufficient* for generalization, although not necessary, e.g., due to the implicit regularization of gradient methods (Soudry et al., 2018). Zhang et al. (2016) argue that while weight decay improves test accuracy, the improvement is not substantial (≈ 1 -2% on ImageNet), indicating the key role of implicit regularization. Loshchilov & Hutter (2019) highlight the distinct effects of weight decay and ℓ_2 regularization, particularly for Adam, suggesting that Adam combined with weight decay (AdamW) leads to superior regularization and simpler hyperparameter tuning. For GPT-3 training, Brown et al. (2020) suggest that they include weight decay to provide *a small amount of regularization*, although we believe it is not the primary reason as we discuss in Sec. 3.

Multiple works have focused on weight decay as a tool influencing optimization dynamics. Van Laarhoven (2017) emphasizes that weight decay’s impact on scale-invariant networks is primarily seen in terms of an effective learning rate. Zhang et al. (2018) propose three mechanisms of weight decay regularization: (1) increasing the effective learning rate for scale-invariant networks, although as we discuss, the same holds for networks beyond scale invariance (2) approximating the regularization of the input Jacobian for an optimizer inspired by second-order methods, (3) inducing a specific dampening effect in this optimizer. Li & Arora (2019); Li et al. (2020) explore the optimization properties of scale-invariant deep networks for which the effective learning rate can be derived. Lewkowycz & Gur-Ari (2020) suggest that the best generalization is achieved with the smallest λ although it necessitates longer training. Additionally, Lewkowycz (2021) propose a criterion for detecting when to decay the learning rate based on the evolution of the weight norm. Bjorck et al. (2021) explore the effect of decoupling weight decay, especially in the early stage of training. Li et al. (2022a) make BERT architecture scale-invariant to enhance training stability and make it more

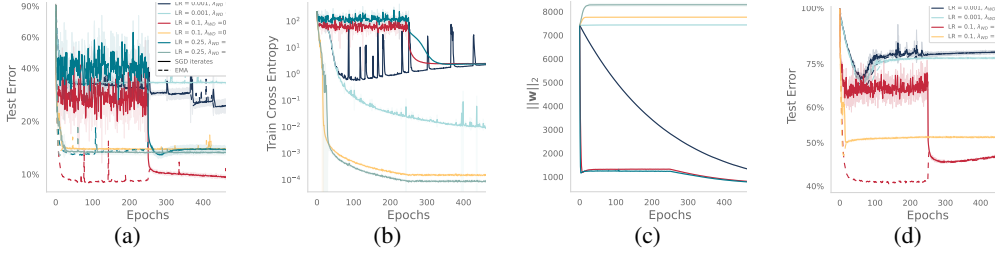


Figure 2: **Training with and w/o weight decay.** We report the test error for Resnet18 on CIFAR-10 (2a) and Tiny-ImageNet (2d) trained with and without weight decay and with small and large learning rates. We also include the correspondent EMA, represented by dashed lines. After the first 250 epochs the learning rate is decayed to $\eta = 10^{-3}$ for all the curves. We report also the L2 norm of the parameters (2c) and Train CE (2b) which after the decay converges to the same value for all the runs with the same λ .

compatible with standard SGD. Recently, Kosson et al. (2023) show a mechanism through which weight decay balances rotational updates across different layers that motivates a new optimizer.

The seminal paper of Krizhevsky et al. (2012) that introduced AlexNet suggest that weight decay serves not only as a regularizer but also reduces the model’s training error, functioning as an *optimization tool*. In recent work, Hoffmann et al. (2022) briefly observe that weight decay enhances the training performance of Adam for training LLMs, but only after $\approx 80\%$ of the total iterations. However, they do not provide an explanation for this behavior, a point we delve into in Sec. 3.

2 Weight decay in the over-training regime

In this section, we delve into the influence of weight decay in the over-training regime, with a specific focus on image classification tasks. We focus on the training of ResNet models (He et al., 2016) using SGD on Tiny-ImageNet (Wu et al., 2017) and report additional experiments in appendix C for VGG, Resnet32 and scale-invariant Resnet architectures on CIFAR10 and CIFAR100 (Krizhevsky & Hinton, 2009).

Notations and setup. Let $(x_i, y_i)_{i=1}^n$ be the training inputs and labels where $x_i \in \mathbb{R}^d$, $y_i \in \mathbb{R}^c$, and c is number of classes. Let $h : \mathbb{R}^p \times \mathbb{R}^d \rightarrow \mathbb{R}^c$ be the hypothesis class of neural network and for any parameter $\mathbf{w} \in \mathbb{R}^p$ where the function $h_{\mathbf{w}}(\cdot) : \mathbb{R}^d \rightarrow \mathbb{R}^c$ represents the network predictions. The training loss $\mathcal{L}$ and the ℓ_2 -regularized loss $\mathcal{L}_\lambda$, for $\lambda \geq 0$, are given by:

$$\mathcal{L}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \ell(y_i, h_{\mathbf{w}}(x_i)) \quad \mathcal{L}_\lambda(\mathbf{w}) = \mathcal{L}(\mathbf{w}) + \frac{\lambda}{2} \|\mathbf{w}\|^2.$$

where $\ell(\cdot, \cdot) : \mathbb{R}^c \times \mathbb{R}^c \rightarrow \mathbb{R}$ denotes the cross-entropy (CE) loss function. With $i_t \sim \mathcal{U}([N])$, the SGD algorithm on $\mathcal{L}_\lambda$ (here with batch size 1 and with replacement) with a learning rate (LR) η is

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla_{\mathbf{w}} \ell(y_{i_t}, h(\mathbf{w}_t, x_{i_t})) - \eta \lambda \mathbf{w}_t. \quad (1)$$

Along the training we track three different iterates: (1) **large-LR** denoted by $\mathbf{w}_t$ which use a large constant LR to exploit the SGD noise, (2) **fine-tuning** $\tilde{\mathbf{w}}_t$ which starting from $\mathbf{w}_t$ use a small LR and (3) the **exponential moving average** (EMA) $\bar{\mathbf{w}}_t$ along the large-LR iterates.

2.1 Loss stabilization and weight decay

To understand whether minimizing the regularized objective $\mathcal{L}_\lambda$ alone ensures optimal generalization, we compare test errors in Fig. 2a across various settings. Although both large and small LR minimize the regularized objective, the evidence that optimal performance is achieved exclusively with large LR indicates that *the objective alone is insufficient to explain the benefits of WD or ensure generalization.*² This experiment reaffirms the widely acknowledged consensus that *implicit regularization induced by the LR is crucial* (Keskar et al., 2016; Li et al., 2019; Andriushchenko et al.,

²The red, blue and green curves have the same CE 2b and norm 2c, hence same $\mathcal{L}_\lambda$ but different test error 2a.